Advanced Computer Architecture

Chapter #2

Memory Technology

Ch#2 Memory technology

Hierarchical Memory Structures,

Memory capacity planning

Cache Memories and Management

Virtual memory technology

Page replacement algorithms

- FIFO algorithm
- Least recently used algorithms(LRU)
- Most frequently used algorithm (MFU)

Memory Devices
and
Hierarchical Memory Structures

Memory

An essential component in any digital computer that is used for storing programs and data

Different types of memories are available

Main memory:- The memory unit that communicates directly with the CPU for program & data

Auxiliary memory:- Besides main memory there are additional storage (having large size and low cost)

Various types of memory forms the memory hierarchy consisting of storage devices employed from the slow with high capacity to fastest with small size

Main Memory

The memory unit that is directly accessed with the CPU for program & data

Used to store programs and data

Can be classified as:

- Read-only memory (ROM) and
- Random-access memory (RAM)

Read Only Memory (ROM)

Consists of a number of memory locations & in each location typically 8-bits are stored like a RAM

Uses random access method just like a RAM

Non-volatile memory so content will not be lost even when power is down

Only read operation is possible from a ROM so it cannot be written during its normal operation

ROM is used to store the content which should not be lost when power is switched off as well which need not to be changed during it normal operation

Different versions of ROM: ROM, PROM, EPROM, EEPROM

Random Access Memory (RAM)

Contains of a various number of memory locations & in each location typically 8-bits are stored

Can be read as well as written

As according to name uses random access method

Volatile memory, the content will be lost when the power is down

Types: SRAM & DRAM

Auxiliary Memory

Auxiliary memory is also frequently known as **secondary** memory

The microprocessor can’t directly communicate with the secondary memory

It is a non-volatile memory

Can be used to store the program files as well as large volume of data

Low speed for access but with a large storage capacity and low cost per unit

For example: Hard disk, Floppy disk, Magnetic tape, Optical disk: CD/DVD, Flash drives etc.

Memory

Primary Memory

- Read-Only Memory
  - Permanent: ROM, PROM
  - Erasable: EPROM, EEPROM
- Read/Write (R/W) Memory
  - Static R/W Memory
  - Dynamic R/W Memory

Secondary Memory (Storage)

- Semi Random Access: Disks – Floppy, HDD, CD, DVD
- Serial Access: Magnetic Tape

Memory Hierarchy

Memory is organized into a hierarchy

CPU Register: At the highest level (closest to the processor)

Cache memory: Different levels of cache memory in between CPU and main memory such as L1, L2, L3

Main memory: Usually dynamic random-access memory (DRAM)

Auxiliary memory: Disk and Tape drives

Auxiliary memory can be: Onboard Storage, as well as Offline Storage

Why Memory Hierarchy?

A variety of technologies available having:

- Faster access time, greater cost per bit
- Greater capacity, smaller cost per bit
- Greater capacity, slower access time

To get better performance at reasonable cost $\Rightarrow$ smaller, more expensive, faster memories are supplemented by larger, cheaper, slower memories

Memory Hierarchy is used to achieve the better performance with minimal total cost of the memory system

Memory Hierarchy

Practically there is a large speed gap between CPU and main memory, i.e. memory is slower than CPU $\Rightarrow$ to close up this speed gap, a hierarchical memory system is used

Memory hierarchy consists of all storage devices employed in a system from slow but high capacity auxiliary memory to a relatively faster main memory, to as even smaller and faster cache memory accessible to the high speed logic

Each memory is involved with different levels in memory hierarchy

Memory Hierarchy

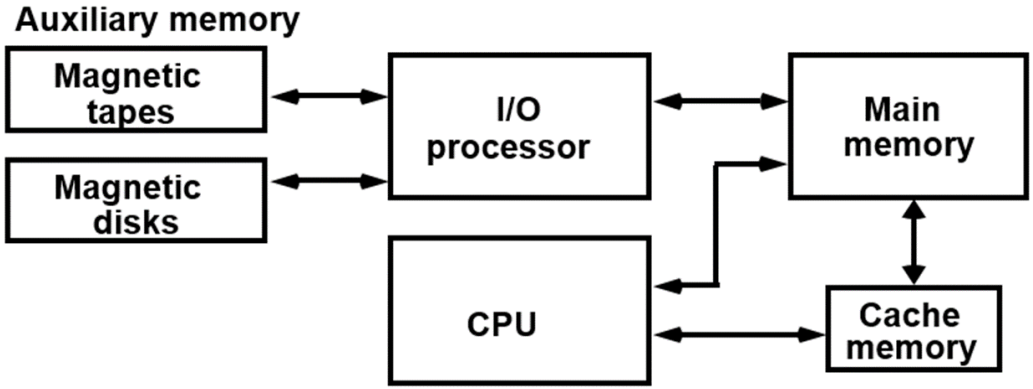
For efficient computer system, it necessary to have a multilevel memory hierarchy

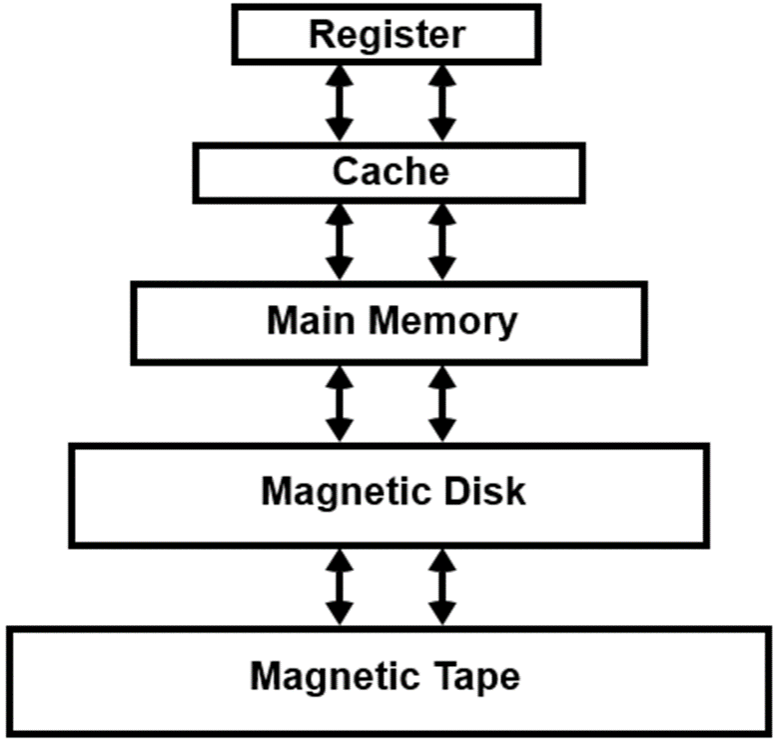
Register is the innermost level of memory ⇒ directly addressable by ALU

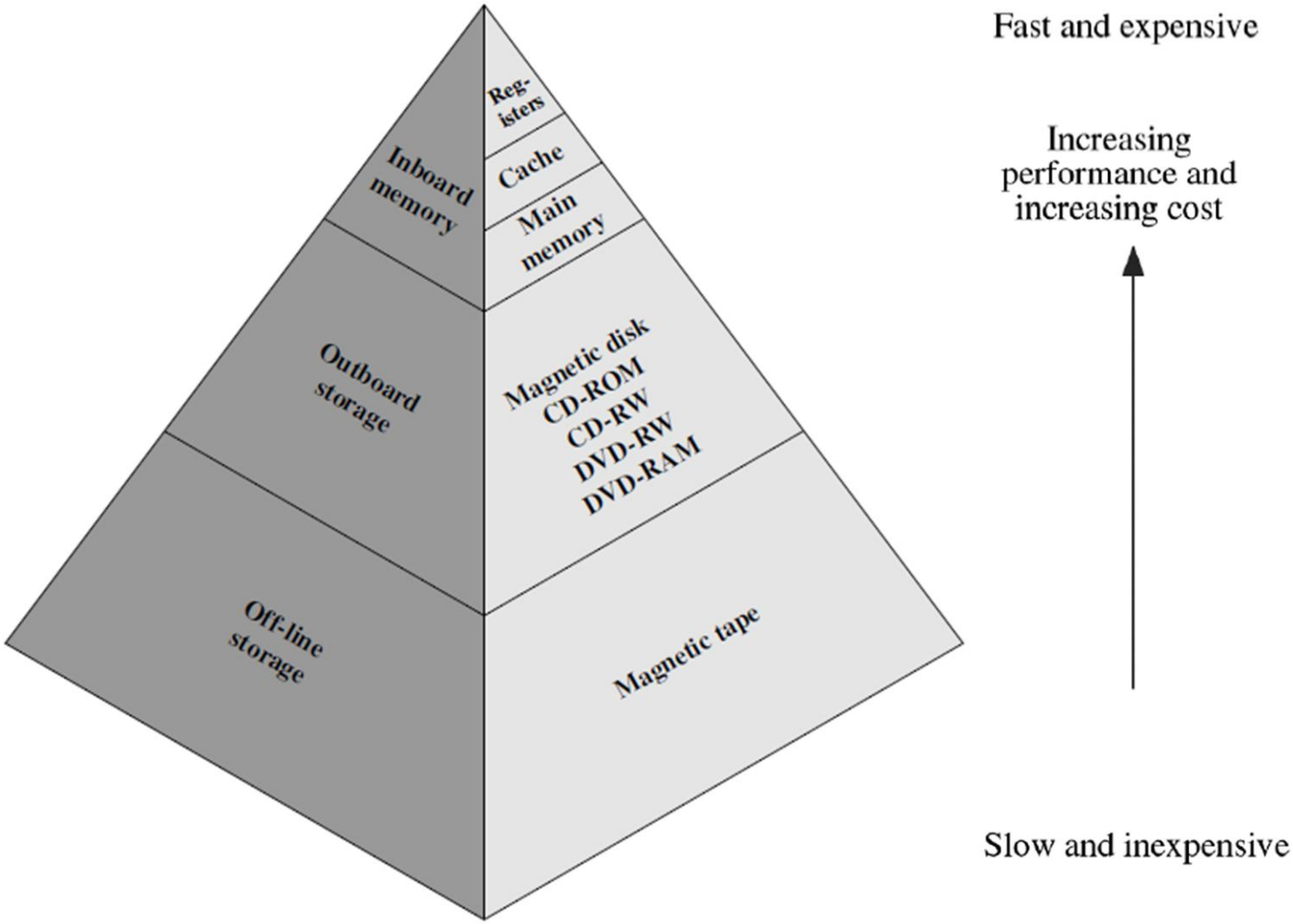
Next layer is cache memory ⇒ acts as buffer between CPU and the main memory

Main memory ⇒ accessed in blocks by using multiway interleaving across parallel memory modules

At the outer levels of hierarchy, disks and tape units are used to establish the virtual memory space


Auxiliary memory
Magnetic tapes
Magnetic disks
I/O processor
Main memory
CPU
Cache memory


Register
Cache
Main Memory
Magnetic Disk
Magnetic Tape


Fast and expensive
Increasing performance and increasing cost
Slow and inexpensive
Inboard memory
Outboard storage
Off-line storage
Reg-isters
Cache
Main memory
Magnetic disk
CD-ROM
CD-RW
DVD-RW
DVD-RAM
Magnetic tape

Parameters Characterize Memory

Access time (t_i): defined as the round-trip time from the CPU to the ith level memory ⇒ can range from a few nanoseconds (ns) to microseconds (μs)

Memory size (s_i): defined as the number of bytes or words in level i

Cost: cost of the ith level memory is estimated as:

$C_i \times S_i$, Where C_i - cost/byte

S_i - No. of bytes/words in level i

Bandwidth (b_i): defined as the rate at which information is transferred between adjacent levels

Unit of transfer (x_i): defined as the grain size for the data transfers between the levels i and i+1

Parameters Characterize Memory

Memory devices at a lower level ⇒ accessed faster, but smaller in size, more expensive per byte, having a higher bandwidth and using a smaller unit of transfer as compared with those at a higher level

Note that,

$$
\begin{aligned}
t_{i-1} &< t_i \\
S_{i-1} &< S_i \\
C_{i-1} &> C_i \\
b_{i-1} &> b_i \\
x_{i-1} &< x_i
\end{aligned}
$$

Where $i \leftarrow O$ to 4

For *e.g.*,
- $i = 0$ means CPU register level
- $i = 1$ means main memory
- ⋮
- $i = 4$ means tape units

$t_i \Rightarrow$ Access time
$s_i \Rightarrow$ Memory size
$C_i \Rightarrow$ Cost
$b_i \Rightarrow$ Bandwidth
$x_i \Rightarrow$ Unit of transfer

Inclusion

Let, **M1** $\Rightarrow$ cache, **M2** $\Rightarrow$ main memory, **M3** $\Rightarrow$ disk storage

According to inclusion property:

$$M_1 \subset M_2 \subset M_3 \subset \ldots\ldots \subset M_n$$

All information is originally stored in the outermost level, M_n

During the processing, subsets of M_n are copied into M_{n-1}

Similarly, subsets of M_{n-1} are copied into M_{n-2} and so on

Inclusion Property

If an information is found in M_i then copies of the same word (8, 16 and 32 bits) will also be found in upper levels M_{i+1}, M_{i+2}, ... M_n

A word stored in M_{i+1} may not be found in M_i

If a word is missing at level $M_i \Rightarrow$ it is also missing from all the lower levels i.e., M_{i-1}, M_{i-2} …. M_1

The highest level is the backup storage where everything can be found

The information transfer between the CPU and cache is in terms of words i.e. 4 to 8 bytes depending on the word length of a machine

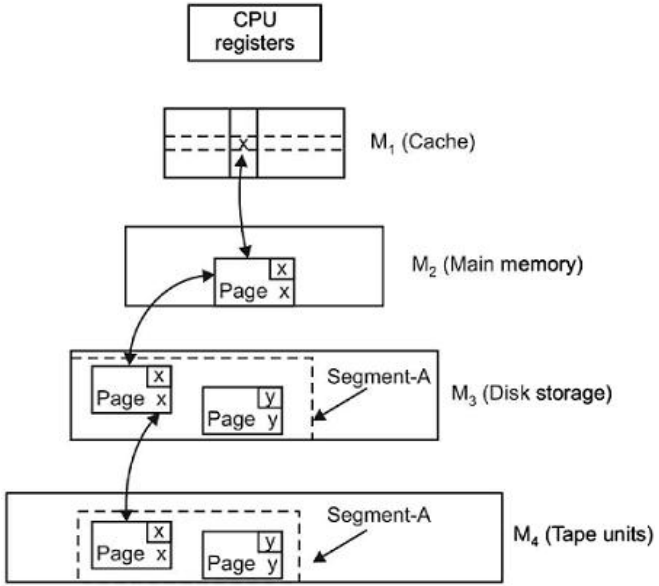


Fig: Inclusion Property

Coherence

Coherence property ⇒ the copies of the same information item at the successive memory level must be consistent

If a word is changed in the cache (M_1-level) then copies of that word must be updated at the higher levels ⇒ but the memory hierarchy is to be maintained

⇒ Store frequently used information at the lower levels (say, M_1) for fast access

Techniques for maintaining the coherence:

- **Write-through (WT)**: an immediate update in M_{i+1} if a word is modified in M_i ($i \leq 1,2, \ldots , n-1$)
- **Write-back (WB)**: delaying the update in M_{i+1} until the word being modified in M_i is replace or removed from M_i

Locality of Reference

According to **Hennessy and Patterson 90-10 rule**: "A typical program may spend 90% of its execution time on only 10% of the code"

Trend of programs where 90% of execution time will be spent in 10% of the code

Improving performance in that 10% region will produce the most significant reduction in execution time ⇒ better performance

For e.g., a program may be spending most of its time on nested innermost loop

Memory addresses are generated by the CPU ⇒ this address is used to access data or instruction

These accesses are actually clustered in certain regions in time, space and ordering ⇒ called as locality of reference

Locality of Reference

Locality of reference or principle of locality $\Rightarrow$ tendency of a processor to access within the small range of memory locations over a short period of time

The references to memory tend to be confined within a localized areas

Types of localities of references:

- Temporal locality
- Spatial locality
- Sequential locality

Temporal locality:

Tends to cluster the access in the recently used areas

e.g., once a loop is entered or subroutine is called, some part of program will be referenced again and again

Locality of Reference

Spatial locality:

Tends to cluster the access of items whose addresses are near to one another

If a word is accessed, adjacent (near) words are likely accessed soon (e.g. Related data items (arrays) are usually stored together; instructions are executed sequentially)

Sequential locality: a special case of spatial locality, occurs when relevant data elements are arranged and accessed linearly

The sequential execution of instructions in sequential order or accessing of an array shows a sequential locality of reference

e.g., accessing elements of a large data array $\Rightarrow$ sequential

Memory Capacity Planning

Memory Capacity Planning

Hit Ratio (h_i)

Hit: When the CPU refers to memory and finds the word in the memory

Miss: It the word is not found in given memory

When required information (instruction or data) is found at given memory (M_i) ⇒ hit, otherwise ⇒ miss

Hit-ratio (h_i) is defined as the probability that an information item will be found in given memory (M_i)

Similarly, miss-ratio at memory (M_i) is ($1-h_i$)

The hit-ratios ⇒ independent random variables ⇒ with values between 0 and 1

$h_i \Rightarrow$ Hit Ratio

$1 - h_i \Rightarrow$ Miss Ratio

The **access frequency** to M_i is given by

$$f_i = (1-h_1)\,(1-h_2)\,\ldots\,(1-h_{i-1})\,h_i.$$

Also $$\sum_{i=1}^{n} f_i = 1$$

Actually, $f_1 \gg f_2 >> f_3 >> \ldots\ldots >> f_n$.

$\Rightarrow$ the inner levels of the memory are accessed more often than the outer levels of the memory

Effective Access Time (T_{eff})

T_{eff} of a memory hierarchy is defined as follows

$$T_{\text{eff}} = \sum_{i=1}^{n} f_i \cdot t_i$$

$$= h_1 t_1 + (1 - h_1) h_2 t_2 + (1 - h_1)(1 - h_2) h_2 t_3 + \ldots +$$

$$(1 - h_1)(1 - h_2) \ldots (1 - h_{n-1}) t_n.$$

T_{eff} depends on the program's behavior and the memory design choices

Total cost of memory hierarchy (C_{total})

Given by the formula

$$C_{\text{total}} = \sum_{i=1}^{n} C_i \cdot S_i$$

The cost is distributed over n levels

Because $C_1 > C_2 > C_3 > \ldots C_n$ so we have to choose $S_1 < S_2 < S_3 < \ldots S_n$

An ideal design of memory hierarchy should have:

$$T_{\text{eff}} \approx t_1 \text{ (of } M_1\text{)}$$
$$C_{\text{total}} \approx C_n \text{ (of } M_n\text{)}$$

But, this is not possible practically due to the tradeoffs among n-levels

Now we have to minimize

$$T_{\text{eff}} = \sum_{i=1}^{n} f_i \cdot t_i$$

and the constraints are:

$$\left.\begin{matrix} S_i > 0 \\ t_i > 0 \end{matrix}\right\} \text{where } i \leftarrow 1 \text{ to } n$$

So,

$$C_{\text{total}} = \sum_{i=1}^{n} C_i S_i < C_0.$$

We need to manage the tradeoffs among t_i, C_i, S_i and f_i (of h_i) at all levels ($i \leftarrow 1$ to n)

Exercise

Consider a three level memory hierarchy (M_1 - M_3) as shown below:

Memory level	*Access time*	*Capacity*	*Cost/KB*
M_1 *(cache)*	$t_1 = 25\ ns$	$S_1 = 512\ KB$	$C_1 = \$1.25$
M_2 *(MM)*	$t_2 = ?$	$S_2 = 32\ MB$	$C_2 = \$0.2$
M_3 *(Disk)*	$t_3 = 4\ ms$	$S_3 = ?$	$C_3 = \$0.0002$

It is desired that:

$$T_{eff} = 10.04\ \mu s$$
$$h_1 = 0.98\ (for\ M_1)$$
$$h_2 = 0.9\ (for\ M_2).$$

But C_{total} <= \$15000

Find S_3, t_2 (marked as ? In the table) ?

Solution

We know that the cost of memory hierarchy is given by the formula:

$C = C_1S_1 + C_2S_2 + C_3S_3 <= \$15000$

or, $1.25\ (512) + 0.2(32) + 0.0002\ (S_3) = 15000$

or, $S_3 = 39.8$ Gbytes

Now,

$$T_{eff} = h_1t_1 + (1 - h_1)h_2t_2 + (1 - h_1)\ (1 - h_2)h_3t_3 \leq 10.04$$

$$\text{or} \quad 10.04 \times 10^{-6} = 0.98 \times 25 \times 10^{-9} + 0.02 \times 0.9 \times t_2 + 0.02 \times 0.1 \times 1 \times 4 \times 10^{-3}$$

$$\text{or} \quad t_2 = \mathbf{903\ ns.}$$

Cache Memory and Management

Cache Memory

Intended to give memory speed approaching that of the fastest memories available using cache

Contains a copy of portions of main memory

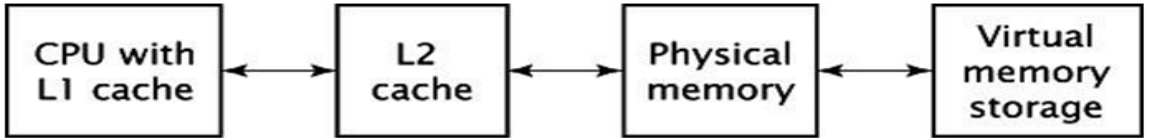
When the processor attempts to read a word of memory, a check is made to determine if the word is in the cache or not

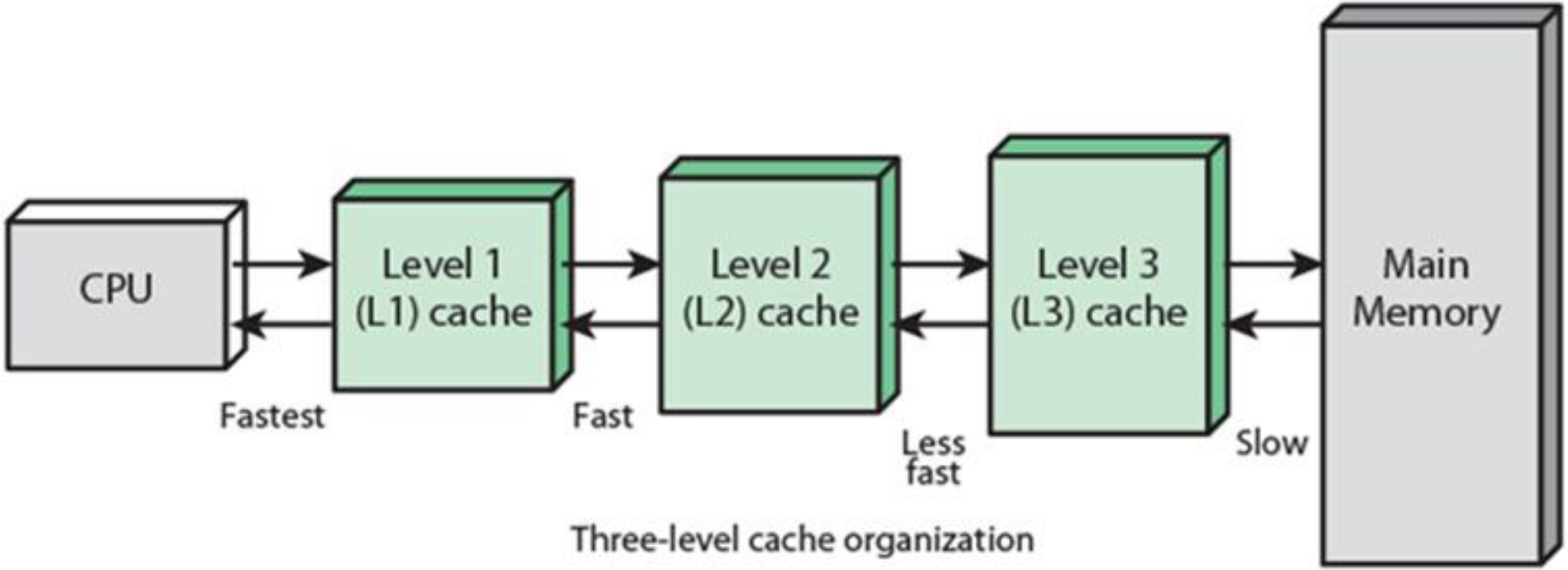
It is small amount of fast memory

Sits between normal main memory and CPU

May be located on CPU chip or module

Memory Hierarchy



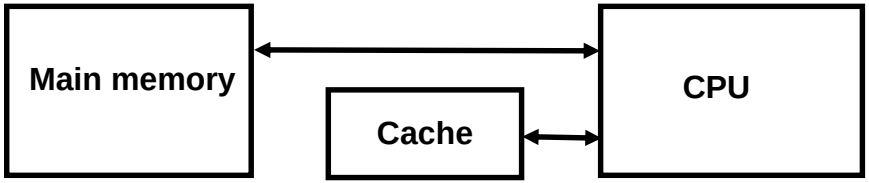


Three-level cache organization

Cache Memory

The property of **Locality of Reference** makes the Cache memory systems work

Cache is a fast small capacity memory that should hold those information which are most likely to be accessed



Performance of Cache

Memory Access

All the memory accesses are directed first to Cache

If the word is in Cache; Access cache to provide it to CPU

If the word is not in Cache; Bring a block (or a line) including that word to replace a block now in Cache

- How can we know if the word that is required is there?
- If a new block is to replace one of the old blocks, which one should we choose?

Performance is measured in terms of **hit ratio**

Cache Memory Principles

Cache memory $\Rightarrow$ fastest memory used in computer

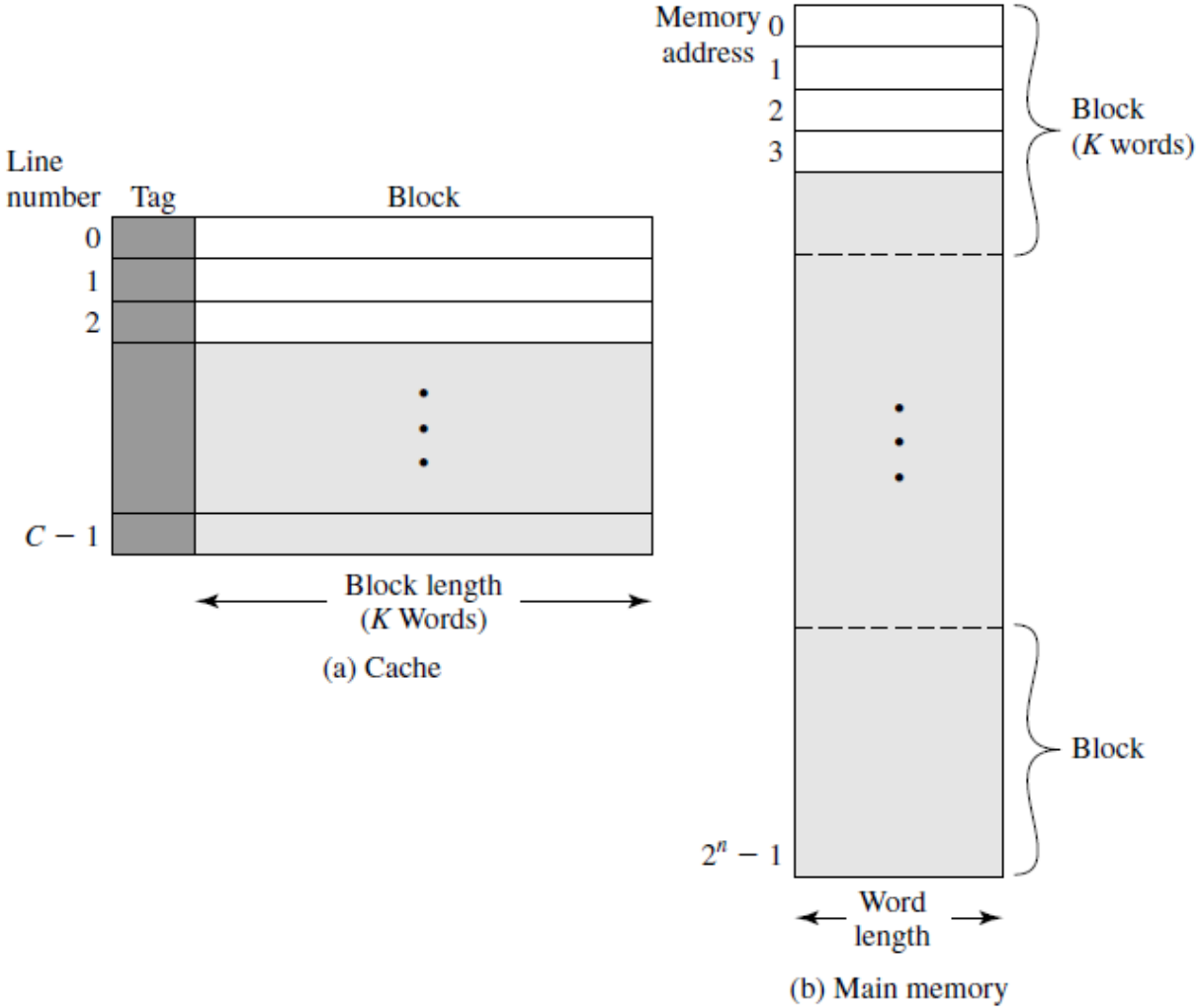
Smaller & faster cache memory backed up with a relatively large and slow main memory

Cache contains a copy of portions of main memory

When the processor attempts to read a word of memory, it checks if the word is in the cache or not

If so, the word is delivered to the processor and if not, a block of main memory, consisting of some fixed number of words, is copied into the cache and then the word is delivered to the processor

Cache/Main Memory Structure



Cache operation

CPU requests contents of memory location

Check cache for this data

If present, get from cache (fast) $\Rightarrow$ hit

If not present $\Rightarrow$ miss, read required block from main memory to cache

Then deliver from cache to CPU

Cache includes tags to identify which block of main memory is in each cache slot

Cache Sizes Examples

Processor	Type	Year of Introduction	L1 cache	L2 cache	L3 cache
IBM 360/85	Mainframe	1968	16 to 32 KB	—	—
PDP-11/70	Minicomputer	1975	1 KB	—	—
IBM 3033	Mainframe	1978	64 KB	—	—
IBM 3090	Mainframe	1985	128 to 256 KB	—	—
Intel 80486	PC	1989	8 KB	—	—
Pentium	PC	1993	8 KB/8 KB	256 to 512 KB	—
PowerPC 601	PC	1993	32 KB	—	—
PowerPC 620	PC	1996	32 KB/32 KB	—	—
PowerPC G4	PC/server	1999	32 KB/32 KB	256 KB to 1 MB	2 MB
IBM S/390 G4	Mainframe	1997	32 KB	256 KB	2 MB
IBM S/390 G6	Mainframe	1999	256 KB	8 MB	—
Pentium 4	PC/server	2000	8 KB/8 KB	256 KB	—
IBM SP	High-end server/ supercomputer	2000	64 KB/32 KB	8 MB	—
Itanium	PC/server	2001	16 KB/16 KB	96 KB	4 MB
Itanium 2	PC/server	2002	32 KB	256 KB	6 MB

Mapping Function

There are fewer cache lines than main memory blocks so mapping algorithm is required

It necessary to know which main memory block currently occupies a cache lines

The choice of mapping function dictates how cache is organized

Techniques for mapping:

- Direct
- Associative
- Set associative

Direct Mapping

Simplest mapping technique

Each block of main memory maps to only one cache line

- i.e. if a block is in cache, it must be in one specific place

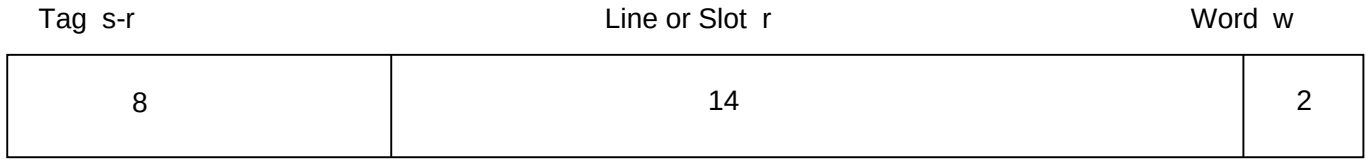
Address is in two parts

Least Significant w bits identify unique word

Most Significant s bits specify one memory block

The MSBs are split into a cache line field r and a tag of s-r (most significant)

Direct Mapping Address Structure



24 bit address

2 bit word identifier (4 byte block)

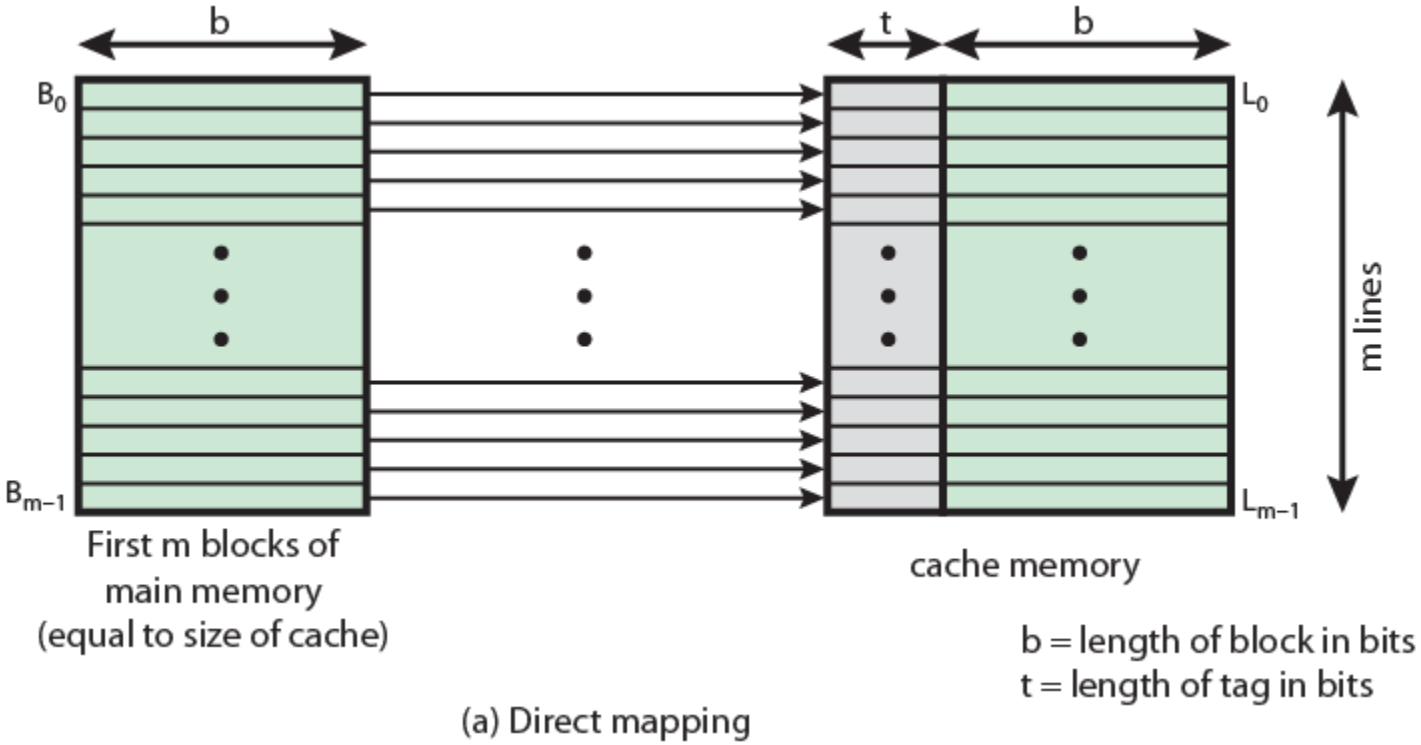
22 bit block identifier

- 8 bit tag (=22-14)
- 14 bit slot or line

No two blocks in the same line have the same Tag field

Check contents of cache by finding line and checking Tag

Direct Mapping from Cache to Main Memory



(a) Direct mapping

Direct Mapping: Pros & Cons

Simple

Inexpensive

Disadvantages:

- Fixed location for given block: If a program accesses 2 blocks that map to the same line repeatedly, cache misses are very high
- The hit ratio can drop considerably if two or more words whose addresses have the same index but different tags are accessed repeatedly

Associative Mapping

Fastest and most flexible cache organization uses associative memory

The associative memory stores both the address and contents (data) of the memory word

Permits any location in to cache to store any word from main memory

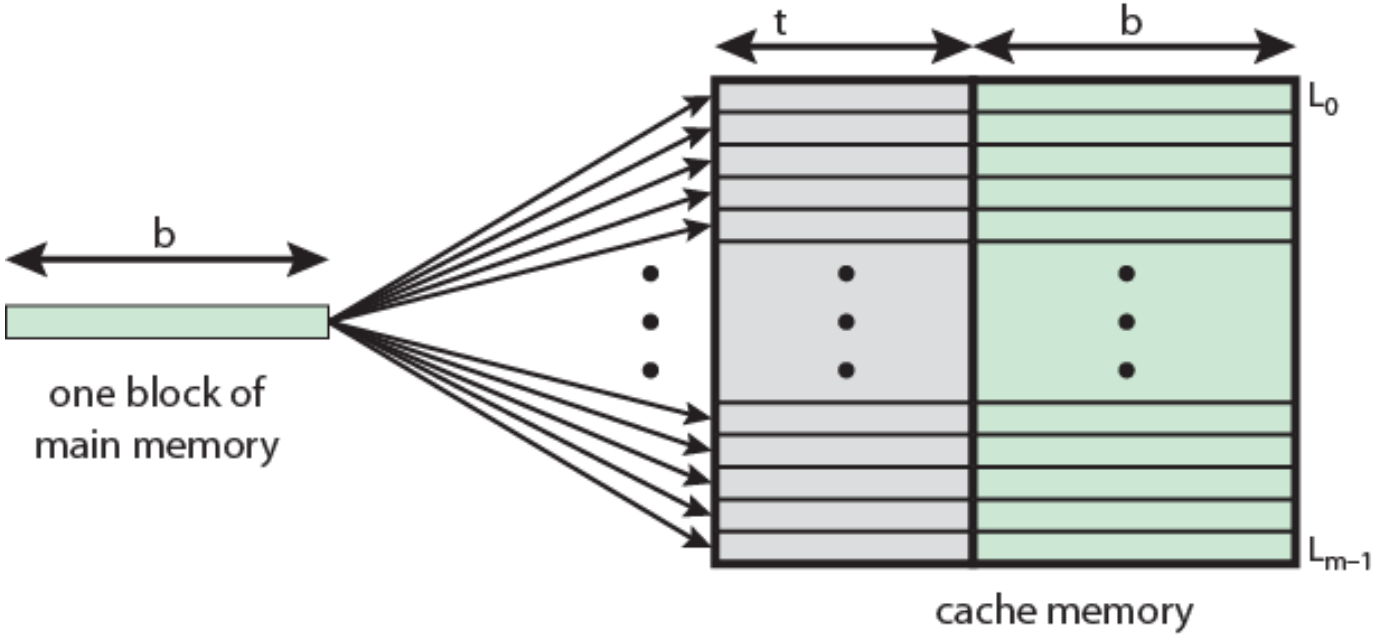
Memory address is interpreted as tag and word

Tag uniquely identifies block of memory

Every line’s tag is examined for a match

Cache searching gets expensive

Associative Mapping from Cache to Main Memory



Set Associative Mapping

In direct mapping technique two words with the same index in their address but with different tag values cannot reside in cache memory at once

So set associative mapping is an improvement over direct mapping

Cache is divided into a number of sets and each set contains a number of lines

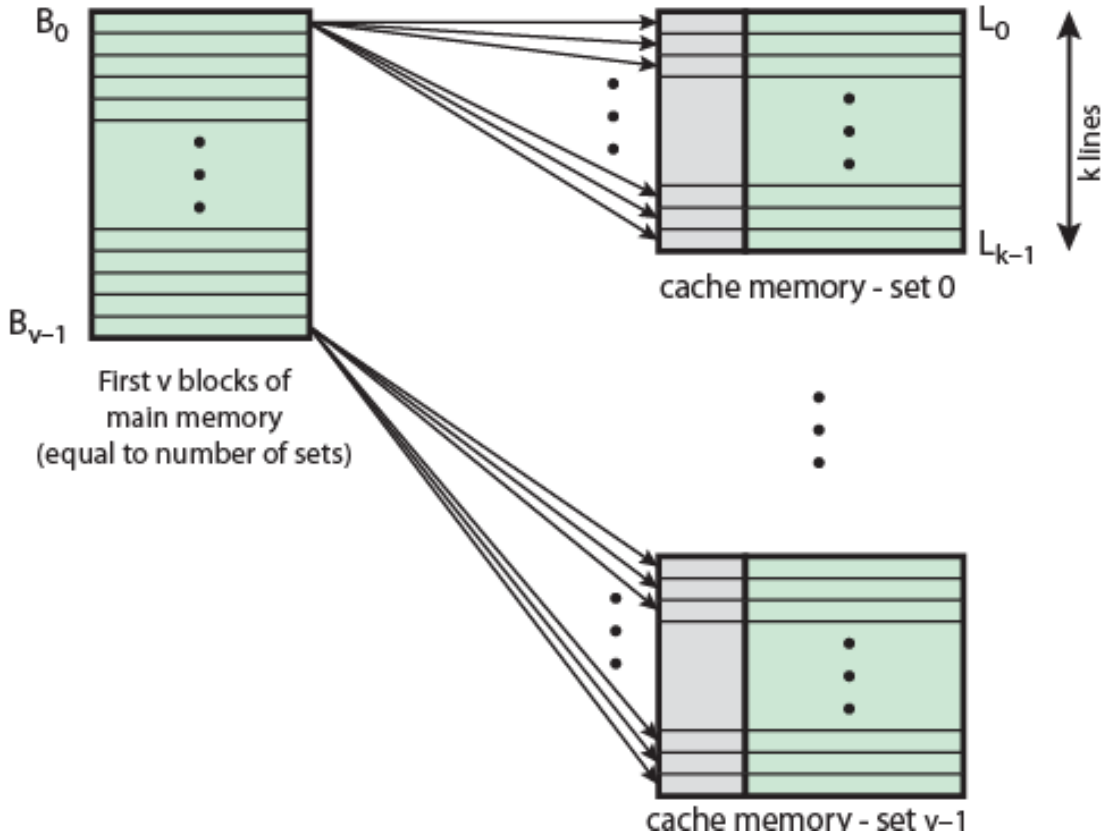
A given block maps to any line in a given set

- e.g. Block B can be in any line of set i

e.g. 2 lines per set

- 2 way associative mapping
- A given block can be in one of 2 lines in only one set

Mapping From Main Memory to Cache: v - Associative



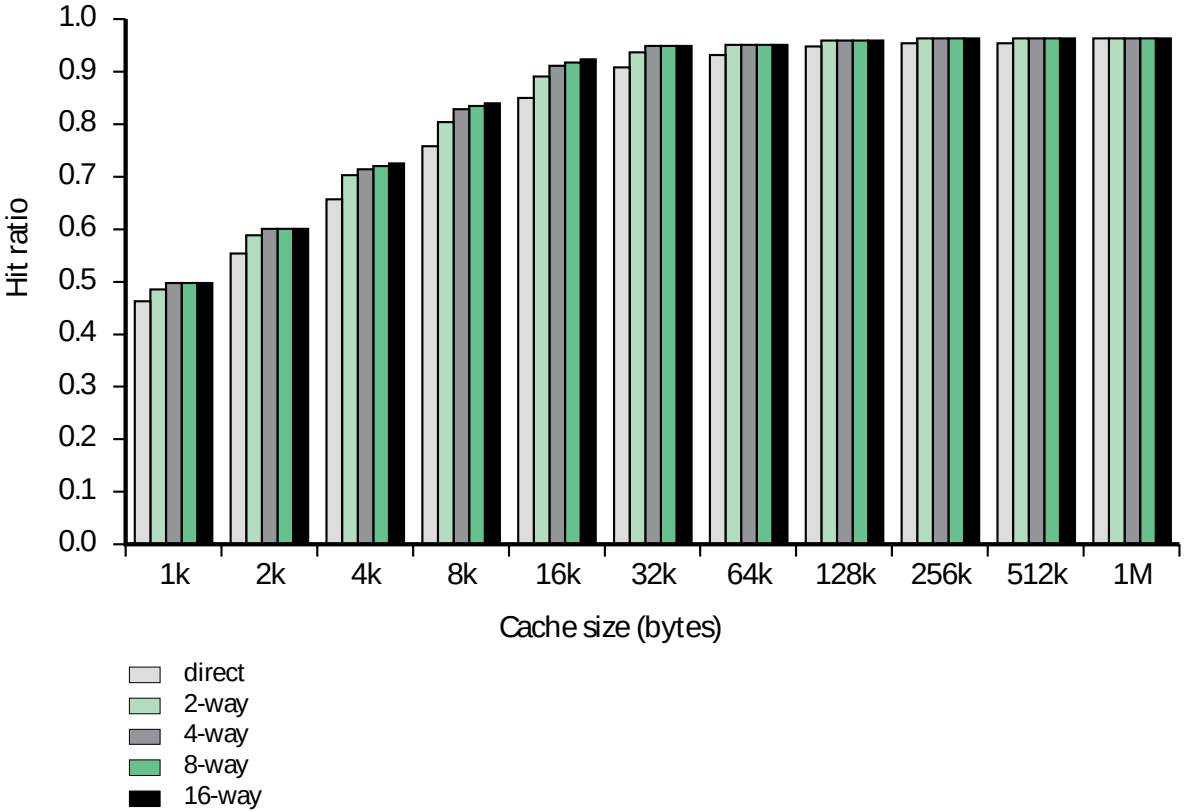
Set Associative Cache Performance

Set associative caches generally have lower miss rates than direct mapped caches of the same capacity

Set associative caches are usually slower and somewhat more complex and expensive to build

Cache complexity increases with associativity

Varying Associativity over Cache Size



Replacement

When a miss occurs and the set is full it is necessary to replace one of the tag-data items with a new value

Different algorithms are available on the basis of cache mapping techniques

Replacement Algorithms: Direct mapping

No choice

Each block only maps to one line

Replace that line

Replacement Algorithms: Associative mapping

A simple procedure to replace cells of the cache is round-robin order

The constitutes a first in first out replacement policy

Replacement Algorithms: Set Associative

Hardware implemented algorithm (speed)

Random replacement

First in first out (FIFO)

- Replace block that has been in cache longest

Least Recently used (LRU), e.g. in 2 way set associative

- Which of the 2 block is LRU?

Least frequently used

- Replace block which has had fewest hits

Unified vs Split Caches

One cache for data and instructions or two, one for data and one for instructions

Advantages of unified cache

- Higher hit rate
  - Balances load of instruction and data fetch
  - Only one cache to design & implement

Advantages of split cache

- Eliminates cache contention between instruction fetch/decode unit and execution unit
  - Important in pipelining

Virtual Memory Technology

Virtual Memory Technology

The main memory ⇒ physical memory

Physical memory ⇒ limited in size

To address the limited size of this memory ⇒ auxiliary memory

Memory hierarchy consists of main memory and auxiliary memory

Auxiliary memory or backup memory ⇒ can be disk arrays

Auxiliary memory can also be used as if primary memory ⇒ virtual memory

Only active programs or some portions of them are stored in physical memory

All programs are loaded in and out of the physical memory dynamically by OS

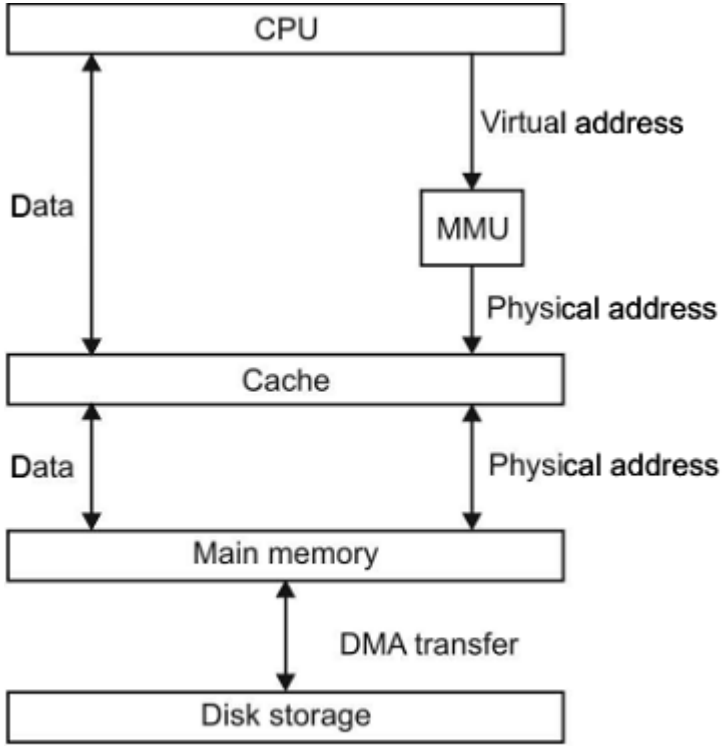
Without virtual memory, it is impossible to use multiprogramming or time sharing

Virtual Memory Technology

Here, the CPU generates virtual addresses that are to be translated to physical addresses at run time ⇒ mapping done by **Memory Management Unit** (MMU)

Ways of having virtual memories:

- Private virtual memory
- Global virtual memory

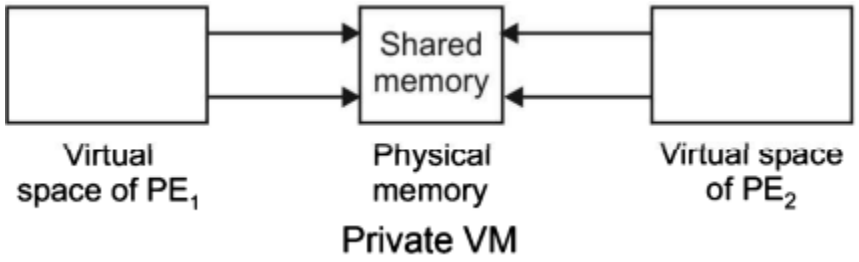


Virtual memory organization

Private virtual memory space model

Each processor has its own private virtual address space

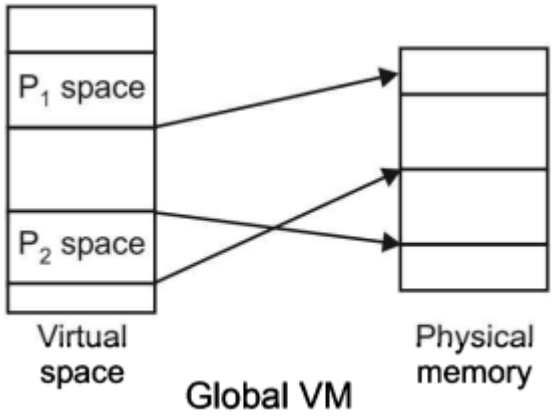
The virtual addresses are mapped to the same physical memory that is shared by all the processors as:



Global virtual memory space model

A single global virtual space is formed

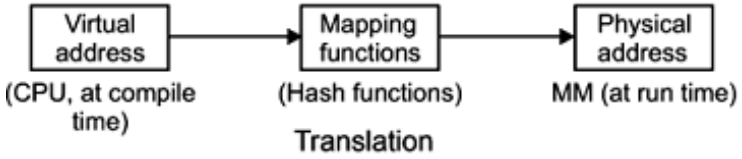
We don't have different spaces for PEs



In this model, all addresses are unique

Role of TLB and PT for address translations

The virtual address (by CPU) are translated to physical address at run time by MMU, as:

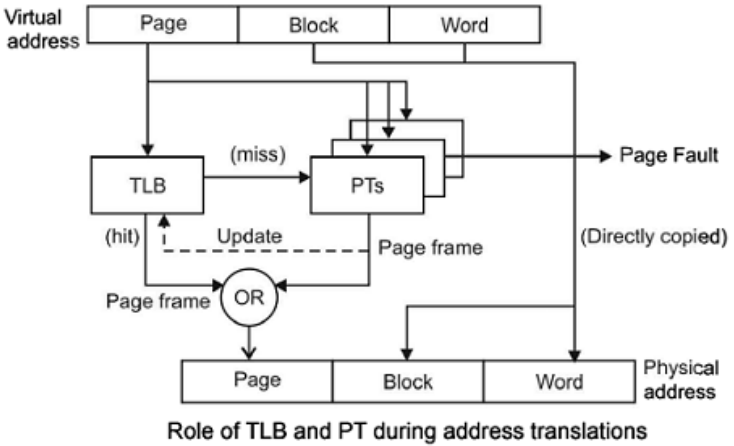


Translation maps are in the form of:

- Translation lookaside buffer (TLB)
- Page Tables (PTs)

On the basis of locality of reference, a particular working set of pages is referenced within a given context or time window

Use of TLB and PT is as:



Role of TLB and PT during address translations

Address Translations

Each virtual address is divided into different fields as:

- Virtual page number
- Cache block number and
- Word address within the block

Main objective ⇒ to produce the physical address having page frame number, block number and word address

Address Translations

Step 1:

The virtual page number is used to search through the TLB for a match

The TLB can be an associative memory or content addressable memory (CAM) or part of cache memory

TLB is a high-speed look-up table ⇒ stores recently referenced page entries

> **Note :** Page entry = virtual page number + page frame number

The pages belonging to the same working set will be directly translated using the TLB entries

Address Translations

Step 2:

If there is a match (or hit) in the TLB, the page frame number is retrieved from the matched page entry

However, the cache block (middle field) and the word address are copied directly

Step 3:

If there is no match (or miss) in the TLB, a hashed pointer is used to identify one of the page table where the desired page frame number can be retrieved

Page Replacement Algorithms

Page Replacement Algorithms

When the processor need to execute a particular page, that page is not available in the main memory ⇒ called as "**page fault**"

Page fault occurs ⇒ page replacement is needed

"**Page replacement**" ⇒ select a victim page in the main memory, replace that page with the required page from the backing store (Disk)

How to select a victim page ⇒ "**Page replacement algorithms**"

The page replacement algorithms select the victim pages

Each operating system has its own page replacement policy

A number of page replacement algorithms are available

Random

Random replacement algorithm

Replaces a random page in memory

As name implies, it randomly purges a page when a new page has to be accessed

No need to keep track of page references $\Rightarrow$ eliminates associated cost of tracking page references

Low overhead $\Rightarrow$ eliminates the overhead cost of tracking page references

FIFO Algorithm (First In First Out)

Simplest algorithm

Idea ⇒ "Replace a page that page is the oldest page of all pages of main memory" or "Replace the page that has been in memory longest"

Focuses on the time a page has been in memory rather than how much the page is being used or how frequently the page is being used

Example: For a memory with 3 frames let us use the reference string as:

0 1 2 3 0 1 2 3 0 1 2 3 4 5 6 7

This example incurs 16 page faults under FIFO

FIFO Example

For given reference string: 0 1 2 3 0 1 2 3 0 1 2 3 4 5 6 7

Table FIFO Behaviour

Frame	0	1	2	3	0	1	2	3	0	1	2	3	4	5	6	7
0	0*	0	0	3*	3	3	2*	2	2	1*	1	1	4*	4	4	7*
1		1*	1	1	0*	0	0	3*	3	3	2*	2	2	5*	5	5
2			2*	2	2	1*	1	1	0*	0	0	3*	3	3	6*	6
	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔

Total 16 page faults in this example

* indicates the new page in the memory

✓ indicates page fault

FIFO Example

The performance of page replacement algorithm is measured through the page fault rate

Page fault rate = Number of page faults/Number of bits in the reference string

= 16/16

= 100%

An algorithm having the least page fault rate $\Rightarrow$ algorithm is the best one

If the required page is in the main memory $\Rightarrow$ page replacement is not required

Optimality: Optimal Page Replacement Algorithm

Optimal page replacement algorithm has the low rate of page faults

Concept: $\Rightarrow$ the page should be removed which will be accessed after the longest time (among the available pages in memory) or replace a page that will not be needed for the longest period of time in the future

If one page will not be used for 8 million instructions and another page will not be used for 6 million instructions, removing the former pushes the page fault that will fetch it back as far into the future as possible

The algorithm relies on perfect knowledge of future page references $\Rightarrow$ unrealistic

Best possible algorithm; easy to describe but impossible to actually implement in a real-time system

Example: Optimal page replacement algorithm

This algorithm has the lowest page fault rate of all algorithms

The criteria for optimality is ⇒ "Replace a page that will not be used for the longest period of time“

This algorithm is difficult to implement, because it requires future knowledge of page references ⇒ this algorithm is used mainly for comparison studies

For a memory of four frames, consider the reference string as:

1 2 3 2 5 6 3 4 6 3 7 3 1 5 3 6 3 4 2 3 4 5 1

Table Optimal Behaviour

Frame	1	2	3	2	5	6	3	4	6	3	7	3	1
0	1*	1	1	1	1	1	1	1	1	1	1	1	1*
1		2*	2	2*	2	6*	6	6	6*	6	6	6	6
2			3*	3	3	3	3*	3	3	3*	3	3*	3
3					5*	5	5	4*	4	4	7*	7	7
	✔	✔	×	✔	✔	×	✔	×	×	✔	×	×	

Frame	5	3	6	3	4	2	4	3	4	5	1
0	1	1	1	1	1	2*	2	2	2	2	1*
1	6	6	6*	6	4*	4	4*	4	4*	4	4
2	3	3*	3	3*	3	3	3	3*	3	3	3
3	5*	5	5	5	5	5	5	5	5	5*	5
	✔	×	×	×	✔	✔	×	×	×	×	✔

∴ Page fault = 11/24

LRU (Least Recently Used) Algorithm

Criteria ⇒ "Replace a page that has not been used for the longest period of time"

Strategy ⇒ "Page replacement algorithm looking backward in time, rather than forward"

Let a reference string with three main frames as:

0 1 2 3 0 1 2 3 0 1 2 3 4 5 6 7

$\therefore$ Page fault rate = 16/16 = 100%

Frame	0	1	2	3	0	1	2	3	0	1	2	3	4	5	6	7
0	0*	0	0	3*	3	3	2*	2	2	1	1*	1	4*	4	4	7*
1		1*	1	1	0*	0	0	3*	3	3	2	2*	2	5*	5	5
2			2*	2	2	1*	1	1	0*	0	0	3	3	3	6*	6
	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔

LFU (Least Frequently Used) Algorithm

"Selects a page for replacement, if the page has not been used often in the past" or

"Replace page that has smallest count"

For this algorithm each page maintains a counter, the value of counter that shows the least count will replaced

The reason for this selection is that, an actively used page should have a large reference count, so don't replace the actively used page

The frequency counter is reset each time a page is loaded

LFU Algorithm Example

For a memory with 3 frames, let a reference string as:

0 1 2 3 0 1 2 3 0 1 2 3 4 5 6 7

Table LFU Algorithm

Frame	0	1	2	3	0	1	2	3	0	1	2	3	4	5	6	7
0	0*	0	0	0	0+	0	0	0	0+	01	0	3*	3	3	3	3
1		1*	1	1	1	1+	1	3*	3	1*	1	1	1	1	1	1
2			2	3*	3	3	2*	2	2	2	2+	2	4	5*	6*	7*
	✔	✔	✔	✔	×	×	✔	✔	×	✔	×	✔	✔	✔	✔	✔

∴ Page faults = 12/16

Belady's Anomaly

Phenomenon in which increasing the number of page frames results in an increase in the number of page faults for certain memory access patterns

Generally if the number of frames is increased ⇒ page fault rate will be decreased

But in certain typical cases the page fault rate will be increased with increased page frames

Commonly experienced when using FIFO page replacement algorithm

In FIFO, the page fault may or may not increase as the page frames increase

But in optimal and stack-based algorithms like LRU, as the page frames increase, the page fault decreases

Belady's Anomaly Example

Let the reference string as: 7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

If we apply the FIFO algorithm with 3 frames ⇒ 15 page faults

If we increase to 4 frames ⇒ 12 page faults

But, for the reference string given as: 1 2 3 4 1 2 5 1 2 3 4 5

Using 4 frames ⇒ 10 page faults

While using 3 frames ⇒ 9 page faults

Here while increasing the frames the number of page faults are also increasing ⇒ this result is **most unexpected** ⇒ known as "**Belady's Anomaly**"

Table 4 Frames

Frame	1	2	3	4	1	2	5	1	2	3	4	5
0	1	1	1	1	1+	1	5*	5	5	5	4	4
1		2	2	2	2	2+	2	1*	1	1	1	5
2			3	3	3	3	3	3	2*	2	2	2
3					4	4	4	4	4	3	3	3
	✔	✔	✔	✔	×	×	✔	✔	✔	✔	✔	✔

∴ Page fault for 4 frames = 10/12

With 3 Frames

Frame	1	2	3	4	1	2	5	1	2	3	4	5
0	1*	1	1	4	4	4	5*	5	5	5	5	5+
1		2*	2	2*	1	1	1	1	1	3	3	3
2			3*	3	3*	2	2	2	2	2	4	4
	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔	✔

∴ Page fault for 3 frames = 9/12

Belady's Anomaly was not always true, it is applicable only to FIFO algorithm